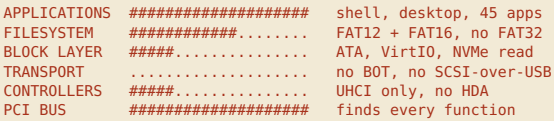
TECHHAVEN STUDIOS

HAVENDOS 1.0 BUILD PLAN

The detailed path from v0.8.1 to v1.0
Seven phases, grounded in the source tree



DOCUMENT THS-RM-1001 REV A · 11 OCT 2026
AUTHOR Kyan Hoekstra / TechHaven Studios
BASELINE v0.8.1, 787 checks across 16 suites
SUPERSEDES THS-RM-1000 sections 6 and 7
STATUS PROPOSAL — no phase agreed or started

1. What changed since THS-RM-1000

That document was written from a reading of the roadmap; this one was written from a reading of the tree, and two of its claims were wrong. Both made the plan look cheaper than it is.

CORRECTION 1

RM-1000 said the layer above the USB controller already existed — that Bulk-Only Transport and SCSI sat on uhci_control() and uhci_bulk(), so an xHCI driver would replace only those two. It does not exist. Nothing in the tree calls either function except shell/usb.c, which only prints a device list. There is no USB block path at all.

CORRECTION 2

RM-1000 implied an AC'97 driver was in place and needed swapping for Intel HDA. There is no audio driver of any kind. The shipping "Music Player" is drivers/speaker.c — PIT channel 2 beeps. Sampled audio starts from zero.

The consequence is an ordering change. RM-1000 put xHCI first on the strength of a stack that turned out to be absent, which made it look like one driver. It is five separate pieces of work. Section 13 reorders the phases accordingly, and the new order reaches a working dual boot sooner.

2. Ground truth

What is actually in v0.8.1, checked file by file. This table is the thing to argue with if any of it looks wrong.

Layer	In the tree	Not in the tree
PCI	Full function-level walk, follows bridges	—
Storage	ATA PIO, VirtIO, NVMe (read only), ATAPI	AHCI, NVMe write, USB storage
USB	UHCI controller, enumeration to a device list	xHCI, EHCI, BOT, SCSI, block registration
Filesystem	FAT12, FAT16, RAMFS	FAT32, LFN, exFAT, NTFS
Partitioning	MBR read+write, GPT read	GPT write, ESP handling
Network	TCP/IP, TLS, DHCP, RTL8139, e1000	RTL8168, any 2006+ NIC
Audio	PC speaker beeps	HDA, AC'97, PCM, WAV, MP3
Boot	BIOS GRUB embedded, Multiboot 1+2, UEFI GOP framebuffer	A UEFI loader it can install

NOTE

HavenDOS boots under UEFI already — that is how the HP runs it. What it cannot do is install a system that boots under UEFI, which needs an ESP (FAT32) and a PE32+ loader. Booting and installing are separate problems and only the first is solved.

3. What 1.0 means

Unchanged from RM-1000, and everything below is scoped to this one sentence:

PROPOSED

HavenDOS 1.0 installs itself onto a computer made in the last ten years, boots from it, reaches the network, and does so without damaging whatever was on the machine beforehand.

Four testable claims: installs, boots, networks, harms nothing. Section 14 turns each into a release gate.

4. How the phases are written

Each phase below has the same five parts, so they can be compared and picked up in any order the dependencies allow:

Part	What it answers
EXISTS	What is already there to build on. Named files and functions.
WORK	Numbered items. Each one is a sitting worth of work or close to it.
TEST	How it is proved. A suite that cannot fail proves nothing.
DONE	The acceptance check. Something observable, not "it compiles".
RISK	What could go wrong and what to do about it.

5. Phase A — FAT32

```
SIZE      Medium
DEPENDS   Nothing
UNLOCKS   Phase C (an ESP is FAT32), USB sticks, most modern media
WHY FIRST It is the smallest phase that unblocks the thing you asked for.
```

EXISTS

fs/fat16.c (1180 lines) and fs/fat12.c (534). The cluster walk, directory entry handling and 8.3 name packing in name_to_83() are all reusable. The block layer underneath is already abstracted.

WORK

A1 fs/fat32.c, read path. BPB differences that actually bite: BPB_FATSz16 is zero and the real size is in BPB_FATSz32; BPB_RootEntCnt is zero and the root is an ordinary cluster chain starting at BPB_RootClus, not a fixed region; FAT entries are 32 bits of which only the low 28 are the cluster — mask with 0x0FFFFFFF on every read or chains will look broken.

A2 Type detection by cluster count, not by label. The FAT type is defined by the count of data clusters: under 4085 is FAT12, under 65525 is FAT16, otherwise FAT32. Reading the partition type byte or the "FAT32 " string instead is the classic way to get this wrong, and both are routinely lying on real media.

A3 Write path. Allocate from the FSInfo next-free hint, extend the chain, write the directory entry. FSInfo must be updated or explicitly invalidated — writing clusters while leaving a stale free count is how a volume comes back wrong under Windows. Setting both fields to 0xFFFFFFFF (unknown) is legal and is the safe option if maintaining them is a nuisance.

A4 Long file names. An ESP has them and so does every USB stick. Minimum viable is skipping LFN entries correctly on read (attribute 0x0F) so they do not appear as garbage 8.3 entries; reading them properly is better and is roughly another afternoon. Writing them can wait.

A5 Mount and route. Teach the VFS and disk about a third filesystem, and the installer's partition scan about type 0x0B/0x0C and the EFI System Partition GUID.

TEST

Build the fixture, do not find one. A make target that runs mkfs.vfat -F32 on a scratch image, populates it with known files, and points a native test at it the way tests/test_fat16path.c points at a disk. Then the reverse direction, which is the one that catches real bugs: write from HavenDOS, then run fsck.vfat -n on the image and require a clean report.

> LESSON ALREADY PAID FOR
>
> tests/test_fat16path.c has never run. It skips unless BOOTDISK names an installer-produced image, no such image is in the tree, and the skip notice was being swallowed by a tail -1 in the Makefile — so the suite appeared in the output with no result and nobody noticed. Fixed in this build. The rule it produces: a fixture a suite needs must be generated by the build, or the suite is decoration.

DONE

Mount a FAT32 image made by mkfs.vfat, list a directory with long names in it, read a file and compare it byte for byte with the original, write a new file, and have fsck.vfat report no errors.

RISK

Low, and bounded. The failure mode is a corrupt scratch image, not a corrupt Windows install, because nothing in this phase goes near a real disk. The one trap is refactoring FAT16 into a shared core while doing it: FAT16 works today, and destabilising it to save duplication is a bad trade. Write fat32.c beside it and factor later, if ever.

6. Phase B — The additive installer

SIZE	Large
DEPENDS	Phase A (the ESP is FAT32)
UNLOCKS	Dual boot. This is the phase you actually asked for.
RISK	Highest in the document. It writes to a disk with Windows on it.

EXISTS

fs/install.c, 626 lines, nine steps. It works and it is the wrong shape: step 1 zeroes LBA 0–63 and the last 33 sectors, step 2 writes a fresh MBR partition table. fs/gpt.c reads and validates a GPT including both CRCs. The unified disk API it all sits on is four functions provided by shell/iso_installer.c:

```
extern int      inst_disk_count(void);
extern int      inst_disk_read (int idx, uint64_t lba, uint8_t *buf);
extern int      inst_disk_write(int idx, uint64_t lba, const uint8_t *buf);
extern uint64_t inst_disk_sectors(int idx);
```

FIGURE 6-1. Every disk the installer can see arrives through these four functions. A new controller is wired in here and nowhere else.

WORK

B1 GPT write, additive only. Find free space between existing partitions, claim the first free slot in the existing entry array, and leave every other entry untouched. Order matters: recompute the entry-array CRC first, then the header CRC over the header with its own CRC field zeroed. Write the primary at LBA 1 with entries at LBA 2, then the backup header at the last LBA with its own copy of the entries. Never rewrite the table wholesale and never zero a header, not even briefly.

B2 ESP install. Mount the existing EFI System Partition — the one already there, the one Windows uses — create /EFI/HAVENDOS/ and write into it beside what is already there. Creating a second ESP is legal and is the wrong answer; firmware will usually pick the first and the install will look like it failed.

B3 A UEFI loader to install. This is a separate artifact and the easiest thing to underestimate. HavenDOS embeds BIOS GRUB today (include/grub_disk_embed.h). A UEFI machine needs a PE32+ EFI application. Two routes: build GRUB for x86_64-efi with grub-mkimage and embed that the same way, or write a small loader that

reads the kernel and hands over a Multiboot-shaped structure. The first is far less work and GRUB already knows how to load the kernel.

B4 Boot registration. Start with the removable-media fallback path \EFI\BOOT\BOOTX64.EFI, which needs no NVRAM access and works on any machine — but note it is the path a USB stick uses, so on an internal disk it takes effect only if nothing else claims priority. A real NVRAM boot entry is the correct answer and needs runtime services, so it comes later.

B5 Self-hosting install. Carried over from 0.8 Beta: an installed system cannot install HavenDOS, because the two-pass build embeds the kernel in itself. The fix is to stream /boot/havendos.elf out of the running filesystem instead — a read-at-offset entry point in fat16.c and a streaming variant of write_file() in fs/install.c. The kernel is 631 KB and the heap is 255 KB, so it cannot be done by buffering.

B6 Invert the refusals. Today GPT means refuse (-14). After this phase GPT is the supported path and the refusals change meaning: refuse when there is no free space, when the ESP is full, when the table is unreadable, when the disk is smaller than the install. The refusal machinery stays; only its reasons change.

NOT NEGOTIABLE

Nothing in this phase ships without a test that builds a Windows-shaped image — protective MBR, GPT, ESP, MSR, a large NTFS-sized partition — checksums every sector outside the region the installer is permitted to touch, runs the install, and asserts those sectors are byte-identical afterwards. Write that test before B1, not after. It is the same discipline that verified live mode, where the attached disk's MD5 was identical before and after a full boot.

TEST

Three layers. Unit: CRC recomputation against known-good tables, free-space search against hand-built layouts, entry packing. Image: the byte-identity test above, plus a run where the disk is deliberately full and the installer must refuse. Boot: install into a QEMU image carrying a real GPT and a real ESP, then boot it under OVMF and reach a shell.

DONE

A QEMU disk with Windows-shaped partitioning on it boots HavenDOS from its own partition under OVMF, the other partitions are byte-identical to before the install, and a disk with no free space is refused with a message that says why.

RISK

The highest in the plan, and the reason for the sequencing rule in section 13: this phase is written and proved entirely against image files, where a bug costs a scratch file. Only once it is proved does a real write path to a real NVMe or SATA disk get enabled behind it.

7. Phase C — Disk coverage

SIZE	B-large, C-small
DEPENDS	Phase B must be proved before C2 is enabled
UNLOCKS	Installing on real machines rather than in QEMU

EXISTS

drivers/ata.c — PIO LBA28 and LBA48 on the legacy ports. drivers/nvme.c — 354 lines, admin and IO queues, polled completion, read only by design. The read half of the hard part is done: queues, doorbells, PRPs and phase tags all work and are covered by 95 checks.

WORK

C1 drivers/ahci.c. PCI class 01/06/01, ABAR is BAR5 and is MMIO. Per port: a 32-slot command list, a FIS receive area, command tables, a PRDT. Commands are READ DMA EXT (0x25) and WRITE DMA EXT (0x35) in a Host-to-Device Register FIS. Poll PxCI for completion the way the NVMe driver polls its phase tag; IRQs can come later. Enumerate every implemented port as a separate disk, not just port 0.

C2 NVMe write. A small addition to a driver that already works — opcode 0x01 against the same IO queue as 0x02. Small in code and large in consequence, which is why it is sequenced after Phase B rather than next to C1.

C3 Both wired into inst_disk_* as new DTYPE_ values, and into the drive picker in shell/iso_installer.c which already knows how to label a type and refuse a row.

C4 Two carried-over storage bugs, both one-liners in scope and both currently invisible: ata_detected() probes the primary master only, so a second IDE disk does not exist as far as the installer is concerned; and atapi_init() probes 0x1F0/0x170 only, so update cannot read the CD on anything with AHCI.

> CHECK BEFORE STARTING C1
> AHCI needs physically contiguous, aligned DMA memory — the command list wants 1 KB alignment and the FIS area 256 bytes. Confirm the PMM can do aligned allocation before designing around it; if it cannot, that is a small prerequisite and better found now than halfway through a driver.

TEST

The simulation technique already used for PCI and UHCI, extended. See section 12 — these are MMIO devices and the existing io.h seam does not reach them.

DONE

QEMU with -device ich9-ahci plus a disk: the picker lists it, a file written from inside HavenDOS is readable from the host image afterwards. Then the same on the real HP over NVMe.

8. Phase D — USB storage

SIZE	Largest in the plan. Five pieces, not one.
DEPENDS	Phase A (sticks are FAT32)
UNLOCKS	Machines with no SATA, USB keyboards, the music player's source

EXISTS

drivers/uhci.c with real transfer primitives — uhci_control(), uhci_bulk(), port reset, a device list, 110 passing checks. That is a third of a USB stack and it is the bottom third. UHCI is USB 1.1, stopped shipping around 2009, and is not on the HP.

WORK

D1 drivers/xhci.c. The controller. Capability registers, then operational registers at CAPLENGTH. A Device Context Base Address Array, scratchpad buffers the controller demands before it will run, a command ring, an event ring with its segment table, a transfer ring per endpoint, and the doorbell array. The cycle bit is the thing to get right: every ring is a circular buffer whose producer toggles a bit on wrap, and a mismatch there produces a controller that silently does nothing.

D2 drivers/usb_core.c. Enumeration that is not tied to a controller: a device handle with a transport behind it, so UHCI and xHCI both satisfy it. GET_DESCRIPTOR walk down device, configuration, interface and endpoint descriptors; SET_ADDRESS; SET_CONFIGURATION. Pick the interface with class 0x08, subclass 0x06, protocol 0x50 — mass storage, SCSI transparent, Bulk-Only.

D3 Bulk-Only Transport. A 31-byte command block wrapper with signature 0x43425355, the data stage, then a 13-byte status wrapper with signature 0x53425355. Tags must match, residue must be honoured, and a stalled endpoint needs CLEAR_FEATURE followed in the bad case by a Bulk-Only Mass Storage Reset. This piece is almost pure logic and is the easiest part of the phase to test properly.

D4 SCSI block commands. INQUIRY (0x12), TEST UNIT READY (0x00), REQUEST SENSE (0x03), READ CAPACITY(10) (0x25), READ(10) (0x28), WRITE(10) (0x2A). Expect a unit-attention sense on first contact and retry rather than failing — a stick that has just been enumerated will report media change once.

D5 Register as a block device so the rest of the system can use it: a DTYPE_USB in the installer's disk table and a route into the FAT mount path. Without D5 the other four are a device list.

TEST

D3 and D4 are structure and state machine work and should be tested exhaustively and natively — wrapper layout, tag mismatch, short residue, stall recovery, sense retry. D1 needs the MMIO seam from section 12 and a simulated controller: ring wrap, cycle-bit toggling, doorbell arithmetic, event dequeue. Then

QEMU's qemu-xhci with a usb-storage device as the integration target.

DONE

A USB stick with a known file on it, plugged into QEMU: usb lists it as mass storage, the installer's picker offers it, and the file reads back byte for byte. Then the same on the HP.

RISK

xHCI is the most complex single device in this plan and the phase has four other pieces behind it. It is also the one most likely to be worth splitting across several sittings with the controller brought up alone first, printing nothing but a port status, before any transfer is attempted.

9. Phase E — RTL8168, and BOOT Chat

> SIZE Medium
> DEPENDS Nothing
> UNLOCKS BOOT Chat, and every other networked thing, on real hardware

EXISTS

Everything above the driver. TCP/IP, TLS, DHCP, the chat protocol, the clients. drivers/rtl8139.c and drivers/e1000.c. The chat is not broken; it has no socket to open.

WORK

E1 drivers/rtl8168.c. Despite the vendor ID it shares with the 8139, this is a different chip with a different programming model: PCIe, descriptor rings for transmit and receive rather than the 8139's single circular buffer, and PHY setup over MDIO. Cover the 8168/8111/8411 family and 10EC:8136 (the 8101/8102 fast-ethernet parts) while in there, since they are the same driver shape.

E2 Nothing else. If the driver presents the same interface as e1000.c, the stack above it does not change and BOOT Chat starts working without being touched.

DONE

ifconfig on the real HP shows a link and a DHCP address, and the chat client connects to the server.

OPEN

Worth confirming before starting: whether that HP has an Ethernet socket at all. Many 2023 15-inch laptops do not, in which case E1 is correct for desktops and other machines but will not make BOOT Chat work on that one, and the honest answer there is Wi-Fi, which section 15 rules out of 1.0.

10. Phase F — Audio

SIZE	Large
DEPENDS	Nothing. Needs no USB, contrary to the original music-player plan.
UNLOCKS	The music player, from zero

EXISTS

drivers/speaker.c, which is a beeper. irq_register() in kernel/idt.c. Nothing else.

WORK

F1 drivers/hda.c. PCI class 04/03, MMIO BAR0. CORB and RIRB command rings, then the part that is genuinely fiddly: enumerate the codec, walk its widget graph, and find a path from an output converter to a physical pin that is connected to something. This varies by machine and is where laptop-specific surprises live.

F2 A buffer descriptor list and a stream descriptor, fed by DMA. There is no scheduler, so playback has to be interrupt-driven off buffer completion with the main loop topping the ring up, and music will stutter whenever an app blocks. That is a known and accepted limitation, not a bug to chase.

F3 WAV. Parse RIFF, feed PCM. Small.

F4 Enable x87 before MP3. The FPU is never initialised anywhere in the tree — no FNINIT, no CR0 setup. It is about twenty lines and it is safe here precisely because there is no scheduler to save FPU state across.

F5 Port minimp3.

DONE

A WAV file plays through the speakers of the real machine without stuttering while the desktop is idle. MP3 after that.

11. Phase G — Finish the console

SIZE	Small, many items
DEPENDS	Nothing
UNLOCKS	The system feeling finished rather than nearly finished

None of these are hard. All of them are visible.

- Desktop apps lay out for 80×25 and draw in the corner of anything larger. 0.8.1 made the console guarantee 80×25 and centre it, which was the cheap half; the apps are the rest.
- The mouse is clamped to column 79 and row 24, so above 80×25 the pointer cannot reach the start button.
- Dotfiles on FAT16 become blank-basename directory entries. This is confirmed, not theoretical — a disk image in the work area holds .SHE and .LOG where .shell_history and .loginlog should be. It affects .accounts, .theme and the rest, which is why the pin files had to be given 8.3-legal names.
- Some themes draw the menu separators invisibly.
- tests/test_fat16path.c needs a generated fixture so it runs at all.

12. How these get tested

Every driver in this plan is a device that cannot be plugged into a test. The tree already solves this once and the solution needs extending.

The seam that exists

include/io.h carries a real include guard as well as its pragma, specifically so a test can define IO_H and supply its own inb/outb. Every port-I/O driver reaches hardware through those six functions and nothing else, so any of them can be compiled natively against a simulated device. That is how PCI enumeration

and the UHCI controller are tested without booting anything — 26 and 110 checks respectively.

The seam that does not

> **PREREQUISITE**
>
> xHCI, AHCI and Intel HDA are all MMIO devices. They do not go through inb/outb, so the existing seam does not reach them, and without a new one all three phases are untestable except by booting. Propose include/mmio.h with the same guard trick — mmio_read32, mmio_write32 and their 16- and 8-bit relatives — adopted before the first MMIO driver rather than retrofitted into three of them.

The rule that stays

A suite that cannot fail proves nothing. Every new suite gets its assertions deliberately broken once to confirm it notices — the technique that caught a wrong address mask in the framebuffer paging work, where 0xFFE00000 passed only because every test address happened to have bit 21 clear.

13. Sequencing

Phases A, D, E and F are independent of each other. B needs A. C2 needs B proved. Within that, the order below reaches a working dual boot in two phases instead of after the largest one, which is the correction to RM-1000.

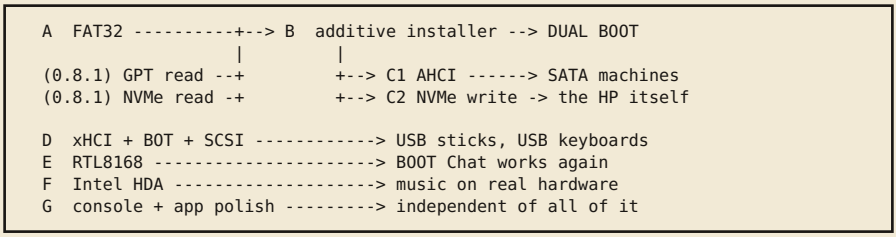


FIGURE 13-1. What blocks what. Only one edge in this diagram is a hard prerequisite: B cannot be finished without A.

SEQUENCING RULE

C2 does not land before B is proved. NVMe write is perhaps forty lines against a driver that already works, which makes it tempting to do early. A write path to a real disk with no additive installer behind it is a loaded gun in the tree — the only thing standing between a bug and a wiped Windows install would be the refusals, and those are meant to be the second line, not the first.

Two ways to order the same work

Track	Order	Gets you
Shortest path to your laptop	A → B → C2	HavenDOS installed beside Windows on the HP's NVMe, booting under UEFI
Widest coverage per line	A → B → C1	Every SATA machine from roughly 2008 to 2020, which is most of them

They share the first two phases, so the choice between them can be deferred until B is done, and either way nothing is wasted.

14. Release gates

The four claims from section 3, each turned into something observable. 1.0 ships when all four pass on hardware, not in QEMU.

Claim	Gate
Installs	Installs onto a GPT disk with another OS on it, from the installer UI, without hand-editing anything
Boots	The machine boots the installed system under UEFI with no removable media attached
Networks	`ifconfig` shows a DHCP address and BOOT Chat connects
Harms nothing	Every sector outside the installed partition is byte-identical before and after, proved by checksum, and Windows still boots

Plus the standing ones: the full suite green, the help checker consistent, the build reproducible from the source archive in an empty directory.

15. Deliberately not in 1.0

Left out	Why
Wi-Fi	Signed firmware blobs per chipset and a full 802.11 MAC. Out of reach for a solo bare-metal kernel, and the honest answer is Ethernet or nothing.
NTFS	Reading a Windows partition is not needed in order to live beside one.
Secure Boot	Requires a signed loader. Users can turn it off, and 1.0 need not solve it.
Multitasking	Orthogonal to running on modern hardware, and large. Phase F already says what its absence costs.
exFAT, LFN writing	Reading long names is in. Writing them, and exFAT at all, can wait for a release that needs them.
IRQ-driven storage	Polled works and is simpler to prove. Revisit when something is actually too slow.

16. Open questions

Two from RM-1000, still unanswered, and neither blocks starting Phase A:

1. Which machines did you try to install on, and what happened on each? Four UEFI laptops makes B everything. A mix including older BIOS desktops moves C1 up and means something else is wrong too.
2. Which modern features do you mean? The inventory in section 2 is what the source says is missing. If you meant something specific that is not on it, it is missing from this plan.

One new one, from writing this:

3. Does the HP have an Ethernet socket? It decides whether Phase E fixes BOOT Chat on the machine you are actually testing on, or only on other machines.

THS-RM-1001 REV A · HavenDOS v0.8.1 · TechHaven Studios · 11 OCT 2026
Supersedes THS-RM-1000 sections 6 and 7. Proposal only — no phase has been agreed or started.